DATA UNDERSTANDING

1. Mengecek Ukuran data dan nama kolom

```
import pandas as pd
```

```
df = pd.read_csv('/content/Isvi_Dataset Case Study - dirty_dummy_events.csv')
```

```
df.head()
```

	event_id	user_id	session_id	event_type	event_timestamp	produ
0	9bd8725f-5722-41b7-8d95-a5dafec33a56	482c0ae4-ac6c-4eb3-91fe-b0e63c1c2a5d	98b232db-5bec-41d8-88e1-5d2e98899e29	product_view	2025-09-22 16:35:27	
1	f6dc704c-a88c-4668-b89d-e6b08e7e02d4	ff6c808d-fa75-4202-801f-a076299a2c0f	d4832af2-0aad-489c-8eb6-22d243f5f5c4	page_view	2025-06-04 21:55:04	
2	abb69be8-df83-420b-92f7-149d29a6d32f	78ea2c10-e824-4b3c-b74b-cdb0cbab0610	1f6c5b12-6c79-4b29-bae6-878804723e55	page_view	2025-06-24 2:02:51	
3	76fd2399-0f80-4184-8802-90dac9bf52d3	9c63419c-b3dc-44a6-b8a5-a438e82afee8	ed1c4ded-cf67-463e-b94f-4cebcf62d827	page_view	2025-06-05 12:35:33	
4	26dcc5fa-334c-420c-be1e-a40eb4b9efac	f9aee3bb-2ba4-4bb3-9eec-44ba4664f93b	9a469fb9-6cf4-4818-b0a6-7302b0bd665a	product_view	2025-07-02 12:19:23	

Next steps: Generate code with df | New interactive sheet

```
df.shape
```

```
(10000, 9)
```

```
df.columns
```

```
Index(['event_id', 'user_id', 'session_id', 'event_type', 'event_timestamp',
       'product_id', 'country', 'device', 'channel'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype 
---  ------           --------------  ----- 
 0   event_id         10000 non-null  object
 1   user_id          10000 non-null  object
 2   session_id       10000 non-null  object
 3   event_type       10000 non-null  object
 4   event_timestamp  10000 non-null  object
 5   product_id       4463 non-null   object
 6   country          10000 non-null  object
 7   device           10000 non-null  object
 8   channel          9900 non-null   object
dtypes: object(9)
memory usage: 703.3+ KB
```

2. Cek missing value

```
df.isnull().sum()
```

	0
event_id	0
user_id	0
session_id	0
event_type	0
event_timestamp	0
product_id	5537
country	0
device	0
channel	100

dtype: int64

3. Cek Duplicate

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df[df.duplicated()]
```

event_id	user_id	session_id	event_type	event_timestamp	product_id	countr

```
#menghapus duplicate jika ada duplicate
df = df.drop_duplicates()
```

4. Cek Consistency

```
df["event_type"].value_counts()
```

event_type	count
page_view	2983
product_view	2552
login	1047
add_to_cart	970
search	945
purchase	509
checkout	503
logout	491

dtype: int64

```
df["event type"] = df["event_type"].str.lower().str.strip()
```

5. Cek Data Unique

```
df["event type"].unique()
```

```
array(['product_view', 'page_view', 'purchase', 'login', 'checkout',
       'search', 'add_to_cart', 'logout'], dtype=object)
```

6. Cek Format Tanggal

```
df["event_timestamp"] = pd.to_datetime(df["event_timestamp"],errors="coerce")
```

```
df["event_timestamp"].isnull().sum()
```

```
np.int64(0)
```

```
df.describe()
```

	event_timestamp
count	10000
mean	2025-06-25 10:08:38.889999616
min	2025-03-27 12:14:19
25%	2025-05-12 01:32:10
50%	2025-06-24 16:36:51
75%	2025-08-08 11:38:53.750000128
max	2025-09-23 11:37:29

```
df["event_timestamp"].describe()
```

	event_timestamp
count	10000
mean	2025-06-25 10:08:38.889999616
min	2025-03-27 12:14:19
25%	2025-05-12 01:32:10
50%	2025-06-24 16:36:51
75%	2025-08-08 11:38:53.750000128
max	2025-09-23 11:37:29

dtype: object

DATA CLEANING

```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
```

```
file_path = '/content/Isvi_Dataset Case Study - dirty_dummy_events.csv'
df = pd.read_csv(file_path)
print("Dataset berhasil dimuat. ")
print("Jumlah baris dan kolom:", df.shape)
```

```
Dataset berhasil dimuat.
Jumlah baris dan kolom: (10000, 9)
```

```
df.describe(include="all").T
```

	count	unique	top	freq
event_id	10000	10000	ac72cfe3-68a8-4229-a676-f220df7c62ce	1
user_id	10000	1982	092be0aa-f430-4c40-bbf4-408f8e58d2cf	14
session_id	10000	10000	ba4892bb-2cef-408e-a09f-ce6db599114b	1
event_type	10000	8	page_view	2983
event_timestamp	10000	9994	2025-06-25 3:06:03	2
product_id	4463	499	P1033	22
country	10000	8	ID	2936
device	10000	3	android	4961
channel	9900	6	email	1707

```
missing = pd.DataFrame({"missing_count":df.isnull().sum()
,"missing_percentage":df.isnull().mean()*100})
missing = missing.sort_values(by="missing_percentage",ascending=False)
missing
```

	missing_count	missing_percentage
product_id	5537	55.37
channel	100	1.00
event_id	0	0.00
user_id	0	0.00
session_id	0	0.00
event_timestamp	0	0.00
event_type	0	0.00
country	0	0.00
device	0	0.00

Next steps: Generate code with `missing` New interactive sheet

```
# Cek duplicate
duplicate_count = df.duplicated().sum()
print("jumlah duplicate:",duplicate_count)
```

```
jumlah duplicate: 0
```

```
df[df.duplicated(keep=False)].head(20)
```

event_id	user_id	session_id	event_type	event_timestamp	product_id	countr

```
#menghapus duplicate jika ada
df = df.drop_duplicates().reset_index(drop=True)
print("shape setelah menghapus duplicate:",df.shape)
```

```
shape setelah menghapus duplicate: (10000, 9)
```

```
print(df.columns.tolist())
```

```
['event_id', 'user_id', 'session_id', 'event_type', 'event_timestamp', 'product_
```

```
# membersihkan nama kolom
df.columns = (df.columns
  .str.strip().
  str.lower().
```

```
  str.replace(" ","_"))
print(df.columns.tolist())
```

```
['event_id', 'user_id', 'session_id', 'event_type', 'event_timestamp', 'product_
```

```
#validasi setelah cleaning
print("shape:",df.shape)
print("\nmissing values:")
print(df.isnull().sum())
print("\nduplicates:")
print(df.duplicated().sum())
print("\nData types:")
print(df.dtypes)
```

```
shape: (10000, 9)

missing values:
event_id              0
user_id               0
session_id            0
event_type            0
event_timestamp       0
product_id         5537
country               0
device                0
channel             100
dtype: int64

duplicates:
0

Data types:
event_id           object
user_id            object
session_id         object
event_type         object
event_timestamp    object
product_id         object
country            object
device             object
channel            object
dtype: object
```

```
#simpan dataset Clean
clean_file = "Isvi_Dataset Case Study.csv"
df.to_csv(clean_file,index=False)
print("dataset berhasil disimpan:",clean_f
```

```
dataset berhasil disimpan: Isvi_Dataset Case Study.csv
```

EXPLORATORY DATA ANALYSIS

```
#Ukuran dataset
print("rows",df.shape[0])
print("columns",df.shape[1])
```

```
rows 10000
columns 9
```

```
#Jumlah user
if "user_id" in df.columns:
  unique_users = df["user_id"].nunique()
  print("Jumlah user unik:",unique_users)
else:
  print("kolom user_id tidak ditemukan")
```

```
Jumlah user unik: 1982
```

```
# Jumlah session
if "session_id" in df.columns:
  unique_sessions = df["session_id"].nunique()
  print("Jumlah session unik:",unique_sessions)
else:
  print("kolom session_id tidak ditemukan")
```

```
Jumlah session unik: 10000
```

```
# Jumlah event
print("total event",len(df))
```

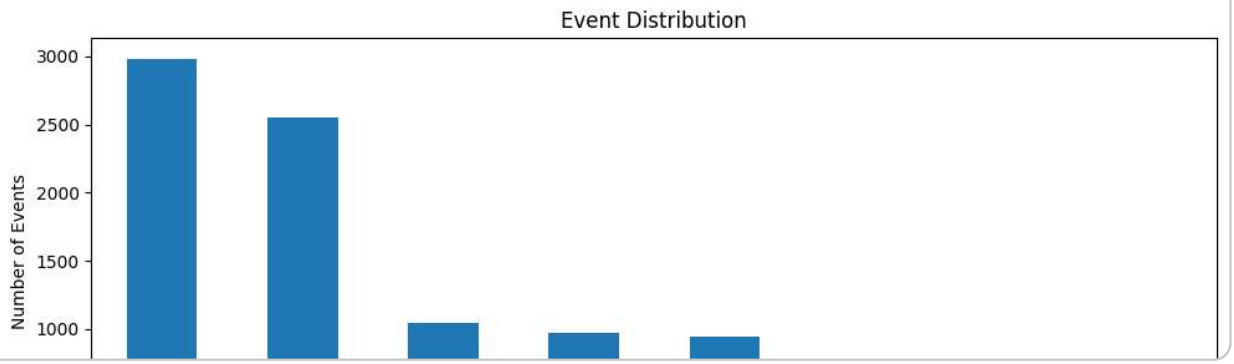
```
total event 10000
```

```
#EVENT DISTRIBUTION
if "event_type" in df.columns:
  event_distribution = (df["event_type"]
                        .value_counts()
                        .reset_index())
  event_distribution.columns = [
      "event_type","event_count"]
```

```
#visualisasi
if "event_type" in df.columns:
  event_distribution.plot(x="event_type",
                          y="event_count",
                          kind= "bar",
                          figsize=(10, 5),
                          legend=False)

  plt.title("Event Distribution")
  plt.xlabel("Event Type")
```

```
plt.ylabel("Number of Events")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



QUETION NO. 1

1. Which country has the most active user in Q2 2025? Make visual comparisons for all country.

```
#cek kolom country
print(df["country"].value_counts())
```

```
country
ID    2936
US    1991
PH    1057
MY    1023
SG    1010
VN    1003
TH     497
AU     483
Name: count, dtype: int64
```

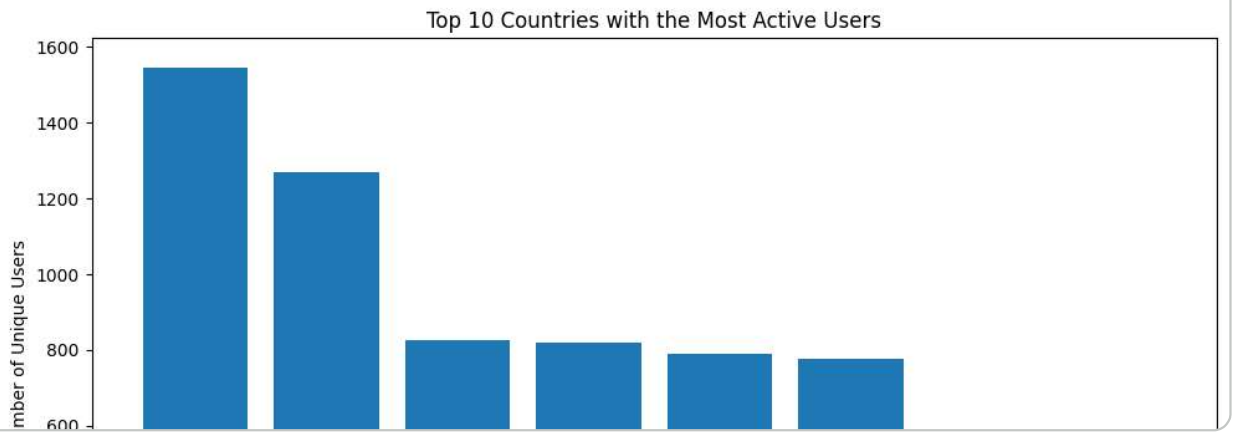
```
#hitung user
users_by_country = (
    df.groupby("country")["user_id"]
    .nunique()
    .reset_index(name="unique_users")
    .sort_values(
        "unique_users",
        ascending=False
    )
)
```

```
users_by_country.head(10)
```

	country	unique_users
1	ID	1546
6	US	1269
3	PH	827
2	MY	819
7	VN	789
4	SG	776
5	TH	436
0	AU	427

Next steps: Generate code with users_by_country | New interactive sheet

```
#VISUALISASI
top_country = users_by_country.head(10)
plt.figure(figsize=(10, 6))
plt.bar(top_country["country"], top_country["unique_users"])
plt.xlabel("Country")
plt.ylabel("Number of Unique Users")
plt.title("Top 10 Countries with the Most Active Users")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



QUETION NO. 2 Monthly Active Users per Channel

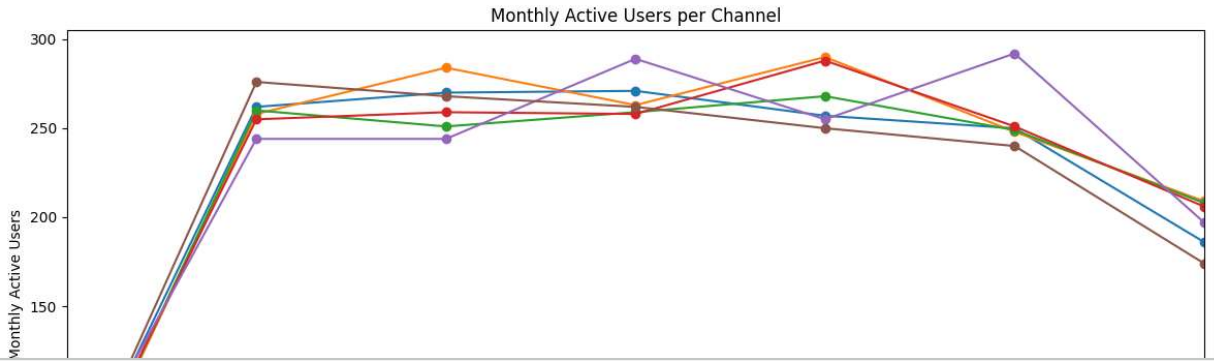
```
# Membutuhkan (event_timestamp, User_id, dan Channel)
# Membuat Kolom Month
# Menghitung Monthly Active Users
```

```
df["event_timestamp"] = pd.to_datetime(df["event_timestamp"], errors="coerce")
df["month"] = df["event_timestamp"].dt.to_period("M")
```

```
#Hitung Mau
mau_channel = (
    df.groupby(["month","channel"])["user_id"]
    .nunique()
    .reset_index(name="Monthly Active Users"))
```

```
#Pivot table
mau_pivot = mau_channel.pivot(
    index="month",
    columns="channel",
    values="Monthly Active Users"
)
```

```
#Visualisasi Mothly Active Users
mau_pivot.plot(
    figsize=(12, 6), marker="o")
plt.title("Monthly Active Users per Channel")
plt.xlabel("Month")
plt.ylabel("Monthly Active Users")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



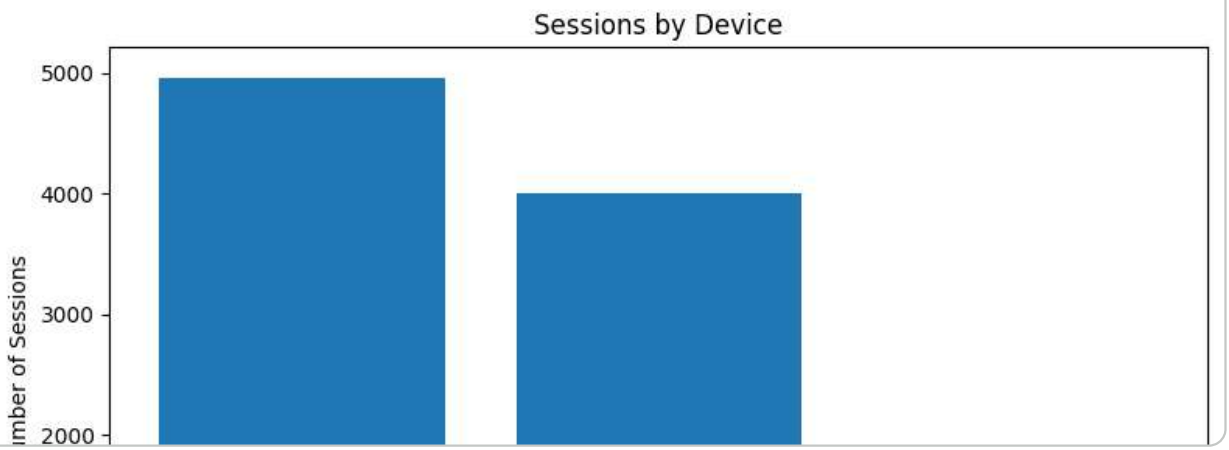
QUESTION NO. 3 DEVICE PER SESSION

```
device_session_df = (df.groupby("device")["session_id"]
                .nunique()
               .reset_index(name='sessions')
                 .sort_values("sessions",
                   ascending=False))
device_session_df
```

	device	sessions
1	android	4961
0	IOS	4000
2	desktop	1039

Next steps: Generate code with device_session_df | New interactive sheet

```
#VISUALISASI
plt.figure(figsize=(8,5))
plt.bar(device_session_df['device'],device_session_df["sessions"])
plt.title("Sessions by Device")
plt.xlabel("Device")
plt.ylabel("Number of Sessions")
plt.tight_layout ()
plt.show()
```



QUETION 4 - SEARCH - Add To Cart

```
#Lihat event
df["event_type"].value_counts()
```

event_type	count
page_view	2983
product_view	2552
login	1047
add_to_cart	970
search	945
purchase	509
checkout	503
logout	491

dtype: int64

```
#session yang melakukan search
search_sessions = set(df.loc[df
 ["event_type"]== "search","session_id"
 ].dropna())
print("Jumlah Session yang Melakukan Search",len(search_sessions))
```

```
Jumlah Session yang Melakukan Search 945
```

QUETION 4a SESSION YANG MELAKUKAN Search dan Add to Cart

```
atc_sessions = set(
    df.loc[
        df['event_type']=="add_to_cart","s
    ].dropna()
)
print("ATC session:", len(atc_sessions))
```

```
ATC session: 970
```

```
search_to_atc = search_sessions.intersecti
print("Sessions with Search then ATC:"),le
```

```
Sessions with Search then ATC:
(None, 0)
```

QUETION NO.5 Corvesion Rate

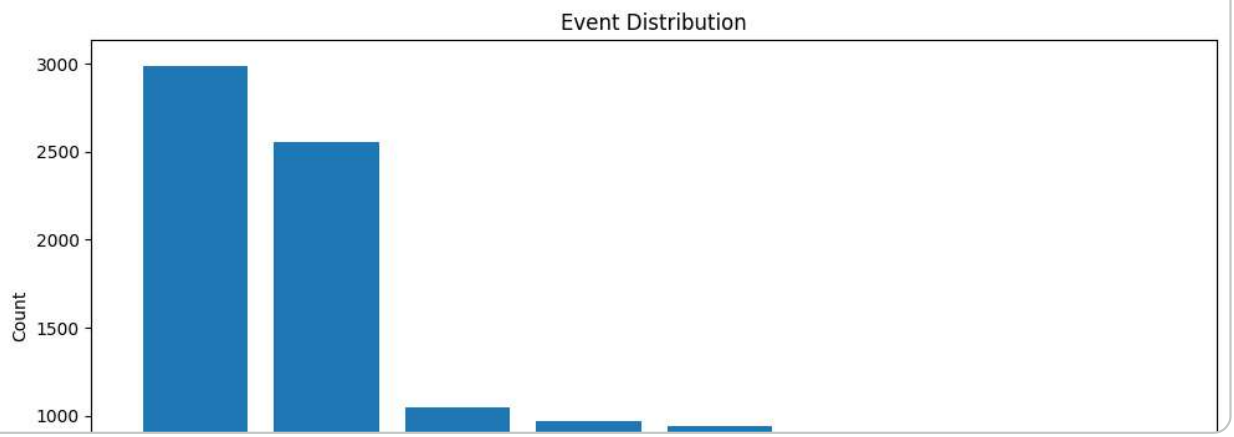
```
search_to_atc_rate = (
```

```
    len(search_to_atc)
    /len(search_sesssions)*100)
print(
    f"Search → ATC conversion rate: "f"{se
```

```
Search → ATC conversion rate: 0.00%
```

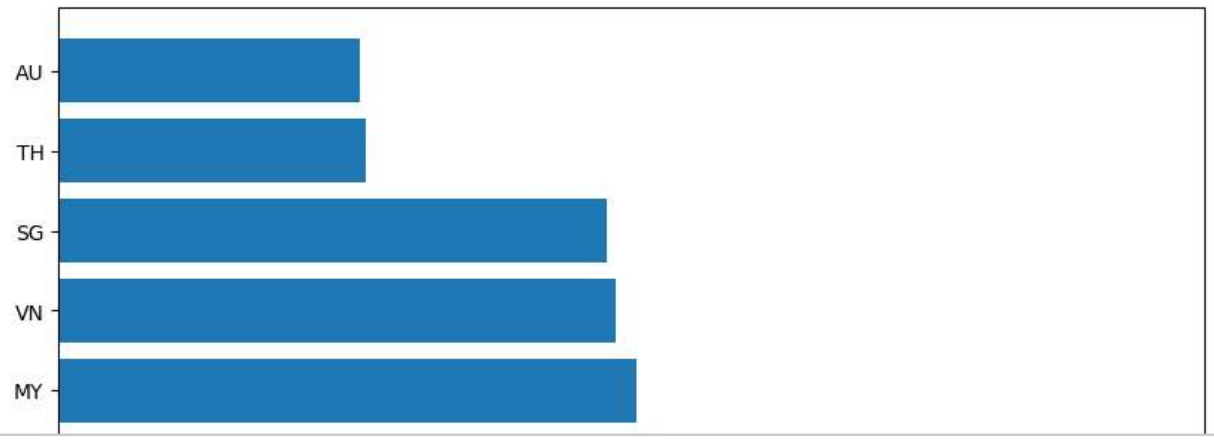
QUETION NO. 5a Visualisasi Event Distribution

```
event_counts = df["event_type"].value_counts()
plt.figure(figsize=(10, 6))
plt.bar(event_counts.index, event_counts.values)
plt.xlabel("Event Type")
plt.ylabel("Count")
plt.title("Event Distribution")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

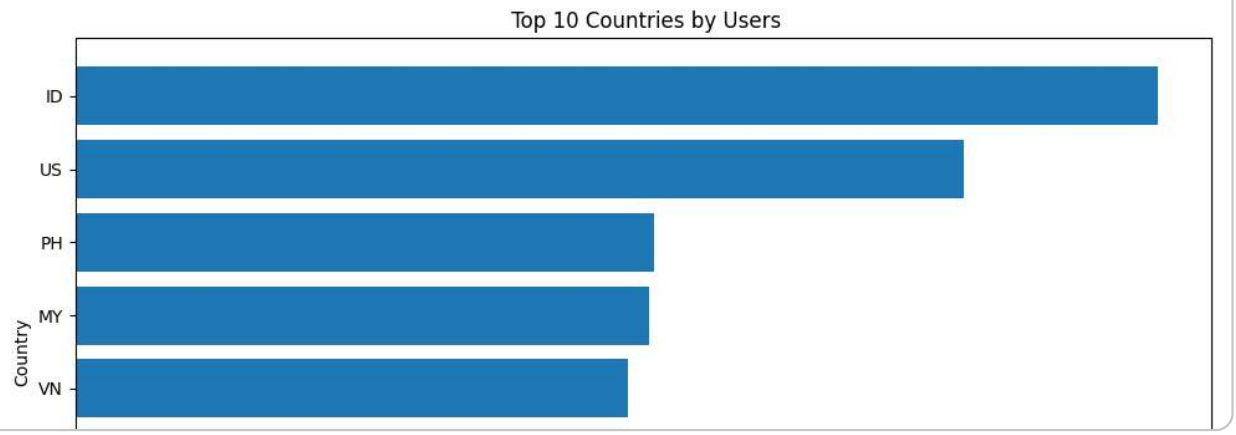



```
#Users by country
top_country = users_by_country.head(10)
plt.figure(figsize=(10, 6))
plt.barh(top_country["country"], top_country["unique_users"])
```

```
<BarContainer object of 8 artists>
```



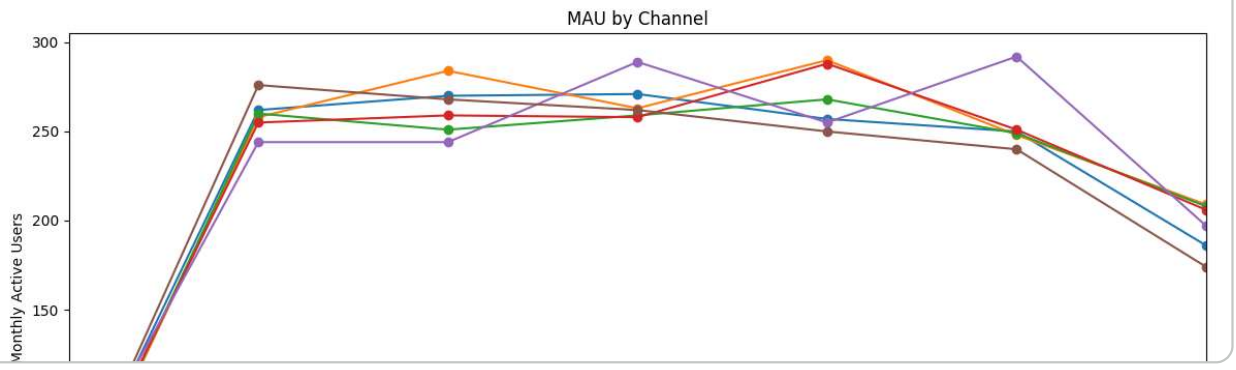
```
#Users by country
top_country = users_by_country.head(10)
plt.figure(figsize=(10, 6))
plt.barh(top_country["country"], top_country["unique_users"])
plt.title(" Top 10 Countries by Users")
plt.xlabel("Number of Unique Users")
plt.ylabel("Country")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```




```
#Monthly Active Users by channel
mau_pivot.plot(
    figsize=(12, 6),
    marker="o"
)

plt.title("MAU by Channel")
plt.xlabel("Month")
plt.ylabel("Monthly Active Users")

plt.tight_layout()
plt.show()
```



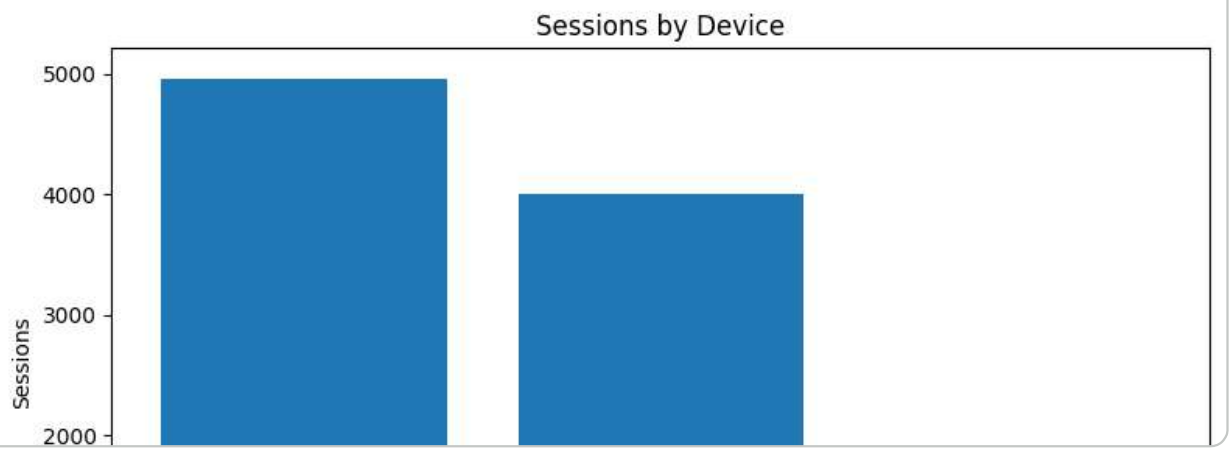

```
#Sessions by Device
plt.figure(figsize=(8, 5))

plt.bar(device_session_df["device"],
```

```
    device_session_df["sessions"]
)

plt.title("Sessions by Device")
plt.xlabel("Device")
plt.ylabel("Sessions")

plt.tight_layout()
plt.show()
```




```
#Search to ATC funnel
funnel_labels = [
    "Search",
    "Search → ATC"
]

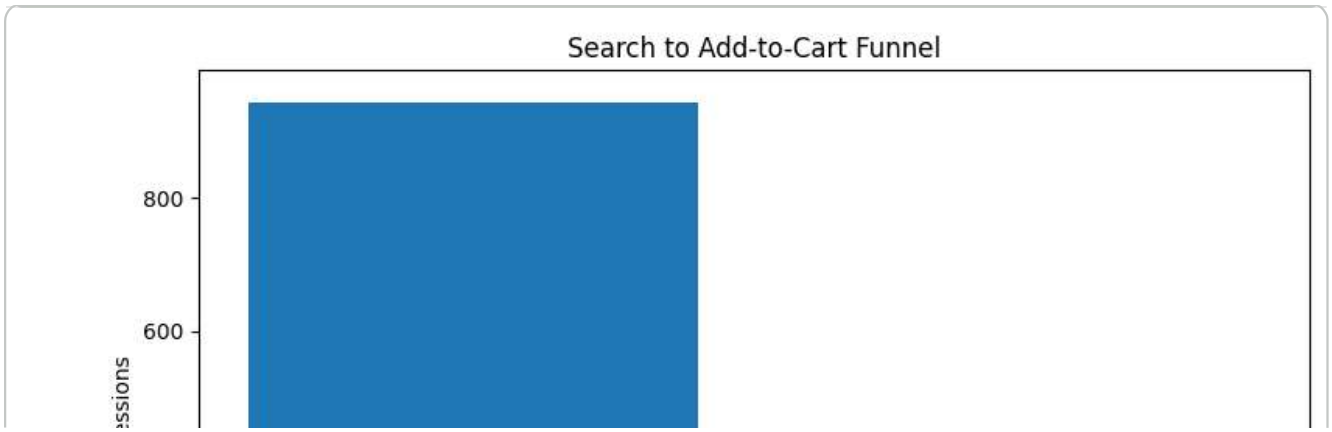
funnel_values = [
    len(search_sessions),
    len(search_to_atc)
]

plt.figure(figsize=(8, 5))

plt.bar(
    funnel_labels,
    funnel_values
)

plt.title("Search to Add-to-Cart Funnel")
plt.ylabel("Sessions")

plt.tight_layout()
plt.show()
```


Search to Add-to-Cart Funnel
800
600